Go Runtime Internals Cheatsheet

L0 Essence: Go Runtime is a self-hosted high-concurrency language runtime leveraging the GMP co-routine scheduler model, an mcache/mcentral/mheap three-level lock-free memory allocation architecture, and a concurrent tricolor mark-and-sweep GC engine with dynamic pacer tuning and hybrid write barriers.

[M1.1] G, M, P Core Structures & Role Division

\textbullet\textbfG (Goroutine): User-space co-routine containing its stack space (starts at 2KB), program counter PC, execution state, and scheduling context (`gobuf` struct preserving `rsp` and `rip`). \\\textbullet\textbfM (Machine): Represents an OS thread managed by the OS kernel, responsible for binding to a P to execute G code. It contains a signal handling stack, a pointer to the active G, and a pointer to the bound P. \\\textbullet\textbfP (Processor): Logical resource processor representing execution context and CPU cores. It holds a local runqueue (`runq` of max size 256) and a thread-local memory allocator cache `mcache`.

[M1.2] Scheduler Main Loop and schedule() Decision Chain

When an M executes the scheduler loop `schedule()`, it prioritizes runnable Goroutines via the following steps: \\\textbf1.\\textbfGlobal Queue Check: Every 61 ticks, the scheduler pulls a G from the global queue (`globalrunq`) to prevent global queue starvation. \\\textbf2.\\textbfLocal Queue Check: Pops a G from the bound P's local runqueue `runq` using lock-free CAS. \\\textbf3.\\textbfNetpoller Check: Checks if any I/O-ready G is returned by the Network Poller. \\\textbf4.\\textbfWork Stealing: If still empty, calls `findrunnable()` to steal Gs from other processors.

[M1.3] Work Stealing Load Balancing Mechanism

Work Stealing balances workloads across multiple logical processor queues: \\\textbf1.\When P's local queue is empty, it randomly selects another P and attempts to steal half of its runnable Gs (up to 128) using lock-free CAS. \\\textbf2.\If all P local queues are empty, it checks the global queue, then epoll network events. \\\textbf3.\If no task is found, the M transitions to "spinning" mode. Spinning Ms search for work. If active spinning Ms exceed active P count, excess threads sleep on `mheap.midle`.

[M1.4] Network Poller epoll Multiplexing Mechanism

Netpoller bridges synchronous Go network APIs and asynchronous OS multiplexers: \\\textbf1.\When a G blocks on socket I/O (e.g., `conn.Read()`), the socket fd is registered with the OS multiplexer (`epoll` on Linux, `kqueue` on macOS). \\\textbf2.\The runtime calls `gopark()` to park the G, detaches it from the M, and allows the thread to schedule other Gs. \\\textbf3.\The scheduler loop or a background thread checks `netpoll` via `epoll_wait`. Ready Gs are marked runnable and pushed back to P's runqueues.

[M1.5] Sysmon Background Thread & SIGURG Signal Preemption

`sysmon` is an OS thread running without a P that monitors runtime health: \\\textbf1.\It runs in cycles (varying from 10µs to 10ms) checking for blocked Gs, network readiness, and long-running Gs (>10ms). \\\textbf2.\Prior to Go 1.14, Go relied on collaborative preemption checking `morestack` at function call points. \\\textbf3.\Go 1.14+ introduced signal-based preemption: if `sysmon` detects a G running for >10ms, it sends a `SIGURG` signal to the M. The M's signal handler stops execution, saves registers, inserts an `asyncPreempt` call, and returns the G to the runqueue.

[M2.1] Three-Level Allocation Architecture: mcache, mcentral, mheap

Go adopts a lock-free/fine-grained allocation architecture derived from tcmalloc: \\\textbf1.\\textbfmcache (Thread-Local Cache): Each P holds an `mcache`. Since a P runs on only one thread at a time, memory is allocated from `mcache` without locks. \\\textbf2.\\textbfmcentral (Central Cache): Manages spans (`mspan`) of a specific Size Class. It requires a lock per class to prevent global heap locking. \\\textbf3.\\textbfmheap (Global Physical Heap): Manages virtual memory pages (8KB pages). It supplies spans to `mcentral` and queries pages from the OS via `mmap` when exhausted.

[M2.2] mspan Memory Layout & Size Class Allocation Flow

Go divides heap allocations into 67 distinct Size Classes (from 8B to 32KB): \\\textbf1.\An `mspan` is the basic memory management unit, composed of one or more contiguous 8KB pages divided into slots of a fixed Class size. \\\textbf2.\For small objects (<32KB), `mallocgc()` maps the size to a class and grabs an empty slot from `mcache.alloc[class]`. \\\textbf3.\If `mcache` is full, it fetches an available span from `mcentral`; if `mcentral` is empty, it requests new pages from `mheap`.

[M2.3] Tiny Object Allocator for Small Objects

For tiny allocations (<16B) that do not contain pointers (e.g., small integers, tiny structs): \\\textbf1.\Go utilizes a Tiny Allocator to group multiple small allocation requests into a single 16-byte block inside `mcache`. \\\textbf2.\The Tiny Allocator keeps track of a byte offset. Subsequent allocations just bump the offset (respecting alignment), reducing internal memory fragmentation.

[M2.4] Compiler Escape Analysis Scene Decision

Escape Analysis is a static compiler optimization pass that determines variable allocation locations: \\\textbf1.\The compiler analyzes the AST to track pointer escape paths. \\\textbf2.\\textbfEscape Rules: If a variable outlives its stack frame (returned as pointer, stored in global variables, sent through a channel, wrapped inside interfaces, or is too large/dynamic), it escapes to the heap. \\\textbf3.\Escaped variables are flagged for heap allocation via `mallocgc()`, while non-escaped variables reside on the stack and are freed when the function exits.

[M2.5] Stack Sizing and copystack Contiguous Expansion

Go uses contiguous stacks to avoid the "hot split" issues of segmented stacks: \\\textbf1.\Each G is initialized with a 2KB stack. At each function prologue, the compiler compares the SP register with `g.stackguard0` to check for overflow. \\\textbf2.\If stack space is insufficient, it calls `newstack()`, which requests a contiguous memory block of double the size. \\\textbf3.\`copystack()` copies the stack data to the new memory, traverses stack frames to adjust stack pointers, and releases the old stack.

[M3.1] Tricolor State Machine & GC Phase Transitions

Tricolor marking is the base of Go's low-latency concurrent garbage collector: \\\textbullet\\textbfWhite: Unscanned objects, candidate for garbage collection. \\\textbullet\\textbfGrey: Reached by GC roots, but fields are not yet scanned. \\\textbullet\\textbfBlack: Reached and scanned; all dynamic children are marked grey. \\\textbullet\\textbfGC Phases: \\\textbf1.\`Sweep Termination` (STW): Cleans unswept spans, activates write barriers. \\\textbf2.\`Concurrent Mark` (Concurrent): GC threads mark live references concurrently. \\\textbf3.\`Mark Termination` (STW): Completes final scans and markers. \\\textbf4.\`Concurrent Sweep` (Concurrent): Sweeps white garbage objects, returns pages.

[M3.2] Dijkstra & Yuasa Hybrid Write Barrier

To guarantee tricolor marking correctness concurrently without STW stack rescans, Go 1.8 introduced the Hybrid Write Barrier: \\\textbf1.\\textbfDijkstra Insertion Barrier: Colors newly written pointers grey, preventing black objects from referencing white objects. \\\textbf2.\\textbfYuasa Deletion Barrier: Colors deleted pointers grey, preventing path loss of reachable objects. \\\textbf3.\\textbfGo Hybrid Write Barrier: Combines both. On pointer writes, it colors the overwritten old value and the new value grey. New stack objects are created black. This removes barriers from stack writes, protecting stack performance.

L1 Logical Framework

\textbf1.\\textbfGMP Model & Work Stealing: Goroutines (G) run on OS threads (M) managed by logical processors (P), using local runqueues and Work Stealing to eliminate global lock contention. \\\textbf2.\\textbfEscape Analysis & Stack Copystack: The compiler statically identifies object escape paths; non-escaped variables reside on contiguous co-routine stacks that expand/shrink dynamically, while escaped objects utilize local caches for lock-free heap allocation. \\\textbf3.\\textbfTricolor GC & Hybrid Write Barriers: GC concurrently marks live references using a tricolor state machine, relying on Dijkstra/Yuasa hybrid write barriers to ensure marking safety without stopping application threads, while GC Assist bounds memory allocation. \\\textbf4.\\textbfChannel Synchronization & Interface Devirtualization: Channels coordinate threads via mutex-protected circular queues and sudog waitlists; dynamic interfaces dispatch methods through itab tables optimized by compiler devirtualization.

L2 Data Flow Topology

`\textbullet\G Created` → `P Local Runqueue` → `M schedule()` → `G Run` → `Heap Alloc` → `Escape Analysis? Yes` → `mcache Alloc` → `Span Exhausted` → `mcentral Fetch` → `Channel Send` → `Buf Full? Yes` → `G blocked on sudog` → `M releases P` → `entersyscall` → `M detaches P` → `exitsyscall` → `G to global queue` → `Net I/O` → `Netpoller epoll_wait` → `wake up G` → `GC Start` → `STW Sweep Term` → `Write Barrier Active` → `Tricolor Mark` → `GC Assist Debt` → `Sweep white obj` → `Return memory to OS.`

Trade-off Matrix: Scheduler & Memory

\textbf\textbfScheduler Model: OS 1:1 Thread Model → User-space GMP Model [OS threads handle syscalls natively → but thread switching costs microseconds and stacks are large (8MB); GMP schedules Gs in nanoseconds with 2KB initial stacks, enabling millions of Gs [Sacrifices scheduling isolations to gain high concurrent throughputs]] \\\textbf\textbfGarbage Collection: Generational Copy GC → Concurrent Tricolor GC [Generational Copy GCs have high throughput and no fragmentation → but trigger long STW pauses; Go Tricolor GC runs concurrently via write barriers, keeping STW under microseconds at the cost of memory fragmentation and barrier overheads [Sacrifices GC throughputs to minimize STW latencies]] \\\textbf\textbfMemory Allocator: Global Mutex Heap Allocator → Three-level local lock-free Allocator [Global allocators are simple and reduce metadata fragmentation → but suffer from CPU lock contentions under multi-threading; Go `mcache` binds to P for lock-free small allocations, sacrificing memory metadata overheads [Sacrifices metadata overheads to gain lock-free allocation performance]] \\\textbf\textbfStack Allocation: Linked Segmented Stack → Contiguous stack copy [Segmented stacks allocate split stack segments instantly → but suffer from "hot split" performance drops in loops; Contiguous stacks double stack memory and copy old frames, which is costly per split, but ensures memory continuity [Sacrifices stack resizing costs to eliminate hot splits]]

[M3.3] User GC Assist Rate Limiter

Under high-allocation rates, if Gs allocate heap memory faster than the GC marks live objects: \\\textbf1.\Go triggers GC Assist to prevent OOM. Each G tracks an allocation debt account. \\\textbf2.\When a G triggers `mallocgc()`, the runtime computes its GC debt based on allocation rate. \\\textbf3.\The G is temporarily halted and forced to perform tricolor marking (scan grey objects) until its debt is cleared before returning with the allocation.

[M3.4] Concurrent Sweep Lazy Scanning

During the Concurrent Sweep phase, Go applies Lazy Sweeping to prevent CPU spikes: \\\textbf1.\Instead of scanning the entire heap inside STW, Go marks spans as "unswept" and resumes user thread execution. \\\textbf2.\When a G calls `mallocgc()` and exhausts its local span cache, it sweeps unswept spans to locate free slots. \\\textbf3.\This distributes the sweep cost across allocation events, removing long sweep pauses.

[M3.5] GC Pacer Feedback Self-Tuning

The GC Pacer optimizes GC triggers based on application behaviors: \\\textbf1.\It maintains a trigger ratio (determining next GC startup based on target heap growth ratio, default $GOGC = 100$). \\\textbf2.\It monitors three inputs: GC CPU usage (targeting 25% CPU limit), allocation rate, and scan throughput. \\\textbf3.\If allocation rate spikes or scanning delays, the Pacer lowers the trigger ratio to start GC earlier; otherwise, it postpones GC.

[M4.1] Channel hchan Memory Layout & Buffer Control

Go Channels are represented by the `hchan` struct: \\\textbf1.\\textbflock: Mutex protecting `hchan` fields. All send/recv calls lock this mutex first. \\\textbf2.\\textbfbuf: Circular array storing sent but unreceived elements. \\\textbf3.\\textbfsendx / recvx: Indices tracking current circular array read/write positions. \\\textbf4.\\textbfsendq / recvq: Wait queues holding blocked Gs represented by `sudog` structures.

[M4.2] Channel Wait Queue sudog & Direct Copy Optimization

When a thread blocks on read/write: \\\textbf1.\\textbfSend Block: G1 sends to a full channel. G1 allocates a `sudog`, points `sudog.elem` to the data address, appends it to `hchan.sendq`, and parks via `gopark()`. \\\textbf2.\\textbfDirect Copy: When G2 reads, it detects G1 in `sendq`. G2 copies G1's data directly from `sudog.elem` to G2's target variable, bypassing the circular buffer, and calls `goready()` to resume G1.

[M4.3] Select Block Multiplexing selectgo Compilation

The `select` statement is handled by `selectgo()` at runtime: \\\textbf1.\\textbfRandom Shuffle: It shuffles the order of cases (lock order and poll order) using a pseudo-random generator, preventing case starvation. \\\textbf2.\\textbfLock & Poll: Locks all cases' channels according to lock order. It then polls each case; if any is ready, it performs the I/O and unlocks. \\\textbf3.\\textbfParking: If no case is ready, it binds a `sudog` to each case, unlocks, and blocks. When any channel wakes it, it cleans other pending `sudog` nodes.

[M4.4] Mutex Normal vs Starvation Modes

Go `sync.Mutex` transitions between two modes to optimize CPU lock throughput: \\\textbullet\\textbfNormal Mode: Waiters queue FIFO. However, arriving Gs spinning on CPU compete for ownership. Spinning Gs usually win. \\\textbullet\\textbfStarvation Mode: If a waiter waits for >1ms, Mutex switches to starvation mode. The lock is handed directly to the queue head; arriving Gs do not spin and append to the queue tail. \\\textbullet\\textbfDowngrade: If a waiter is the last in queue or waits for <1ms, it switches back to Normal.

[M4.5] WaitGroup & Once Sync Primitives

\textbullet\\textbfWaitGroup: Composed of a task `counter` and a semaphore `sema`. `Add()` updates the counter. `Wait()` appends the caller to `sema` if the counter > 0. `Done()` decrements the counter; when it reaches 0, all blocked callers are awakened. \\\textbullet\\textbfOnce: Contains a `done` flag and a `Mutex`. `Do(f)` calls `Load` on `done` first. If `done` is 0, it locks, checks again (Double Check), executes `f()`, and atomics `done` to 1.

[M5.1] Map hmap & bmap Memory Layout

Go Map is a hash table using buckets: \\\textbf1.\\textbfhmap: Master header containing bucket array pointer, count, and overflow buckets. \\\textbf2.\\textbfbmap (bucket): Holds up to 8 key-value pairs. It stores 8 `tophash` values (high 8 bits of hashes), then 8 keys, and then 8 values. This layout avoids paddings. \\\textbf3.\\textbfOverflow: If keys in a bucket exceed 8, an overflow `bmap` is linked to the bucket tail.

[M5.2] Map Progressive Rehash Evacuate Mechanism

Maps scale up or reorganize when buckets get cluttered: \\\textbf1.\\textbfDouble Size: Triggered when load factor > 6.5. \\\textbf2.\\textbfSame Size: Triggered when overflow buckets are too many. \\\textbf3.\\textbfProgressive Evacuation: Go relocates old buckets progressively. Each write or delete call evacuates the targeted bucket and one old bucket. Old bucket data is split and copied to the new buckets.

[M5.3] Empty interface eface vs non-empty iface Structure

Go interfaces represent dynamic type containers: \\\textbullet\\textbfeface (empty interface `interface`): Holds any type. Composed of two pointers: `_type` pointing to dynamic type metadata, and `data` pointing to the value. \\\textbullet\\textbfiface (non-empty interface): Composed of `itab` and `data`. \\\textbullet\\textbfitab structure: Stores interface type metadata, concrete type `_type` metadata, and virtual method pointers `fun`.

[M5.4] Interface Dynamic Dispatch & Devirtualization

Interface method invocations require dynamic lookup: \\\textbf1.\\textbfDispatch: Reads `iface.itab`, fetches the method pointer from `fun`, passes receiver `iface.data` to registers, and jumps. This dynamic call has cache misses and cannot be inlined. \\\textbf2.\\textbfDevirtualization: The compiler optimizes interface calls: if the concrete type can be statically determined, it rewrites the call to a static direct call, enabling function inlining.

[M6.1] Defer Compiler Optimization: Heap, Stack & Open-coded

Go optimizes `defer` allocations through three strategies: \\\textbf1.\\textbfHeap Allocated: Allocates `_defer` instance on heap via `deferproc()`, resolved at exit. Heavy cost. \\\textbf2.\\textbfStack Allocated: Allocates `_defer` on stack frames when defer is outside loops. \\\textbf3.\\textbfOpen-coded: When defer counts are fixed and outside loops, the compiler inlines defer calls directly at return paths, using local bitmasks to track execution, achieving zero-cost defer.

[M6.2] Panic Nested Chain & Recover Processing

\textbf1.\\textbf_panic structure: A panic links a `_panic` struct to the G's header, containing a `recovered` flag. \\\textbf2.\\textbfExecution: Go stops normal flows and executes the G's `_defer` chain in LIFO order. \\\textbf3.\\textbfRecover: If a defer calls `recover()`, Go marks `_panic.recovered = true`, reads stack context `g.sched`, and jumps to the defer caller's return path.

[M6.3] Syscall GMP Detach & entersyscall/exitsyscall

\textbf1.\\textbfentersyscall: When calling blocking syscalls, the wrapper detaches P from M. M enters kernel syscall. \\\textbf2.\\textbfsysmon Takeover: `sysmon` detects the blocked M, detaches the idle P, and binds P to another M to execute other Gs. \\\textbf3.\\textbfexitsyscall: On syscall exit, M attempts to acquire its old P or any free P. If none are free, G is sent to the global queue and M sleeps.

[M6.4] Cgo Call Overhead and Stack Switches

C lacks Goroutine scheduling contexts; calling C requires crossing runtime boundaries: \\\textbf1.\\textbfStack Switch: Co-routine stacks are small (2KB). C code requires native thread stacks (8MB). Cgo calls `asmcgocall()` to switch SP to the M's `g0` stack. \\\textbf2.\\textbfLocking: The caller G is marked as syscall state so GC ignores it, and C pointers are pinned. \\\textbf3.\\textbfCost: Switch actions and cross-boundary marshaling slow Cgo calls down relative to native Go.

Zone T1: Go debug flags & tracing

`GODEBUG=schedtrace=1000` : Prints scheduler status summaries every 1000ms, showing global queue size, local queue sizes, and active M/P counts. \\`GODEBUG=gctrace=1` : Prints GC logs on completion, showing heap reclaim ratios, STW durations, and GC CPU allocations. \\`go build -gcflags="-m -l"` : `-m` prints compiler escape analysis outputs; `-l` disables function inlining to simplify backtrace debugs.

Zone T2: Go assembler & write barrier listings

\textbfStack Split Check: \\Go functions insert checks at the prologue to verify stack spaces:

```
\\MOVQ (TLS), CX // Get active Goroutine pointer g     \\CMPQ SP, 16(CX) // Compare SP with g.stackguard0 (offset 16)     \\JLS 24 // Jump to newstack if SP <= stackguard0
```

\\textbfWrite Barrier Check: \\When write barriers are active, pointer modifications trigger barriers:

```
\\LEAQ runtime.writeBarrier(SB), R11 \\CMPB (R11), $0 // Check if write barrier is active   \\JEQ L_direct_write // If 0, execute direct write    \\CALL runtime.gcWriteBarrier(SB) // If 1, call barrier to color references grey
```